

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..2

RUNNING YOUR SERVER

 Copy as Markdown

Running your server

As you build, one command runs your server, database, and app together. When you save a file, it regenerates code and hot reloads your running server and app. The `serverpod start` command generates the latest code first, then runs everything inside a single interactive terminal.

Run it from your project's root folder or one of its package folders:

```
serverpod start
```

The interactive terminal shows a tab for the server and for each running app, with the most common actions along the bottom. Shortcuts are unshifted key presses: **M** means typing a lowercase `m`, while **Shift+M** produces the capital and triggers the variant. Press **H** for the full list of shortcuts, and **Q** to quit the session. If a session is already running for the project, a second `serverpod start` tells you so and exits. While the session runs, it also exposes an MCP endpoint that AI agents can use to drive it. See the [serverpod mcp-server](#) reference.

New projects use an embedded PostgreSQL database that the server manages for you, so there is nothing else to start. See [Database backends](#) for how it is configured and how to use an external database instead.

Use `serverpod start` for local development only. To run your server in production, deploy it instead. See [Deploy to Serverpod Cloud](#) or [Custom hosting](#).

Save a file to hot reload

By default, `serverpod start` watches your project. What happens when you save depends on the file:

- **Server code:** hot reloads, and restarts the server if the reload fails.
- **Models:** regenerates code and hot reloads. The database does not change until you [apply a migration](#).
- **Flutter app code:** hot reloads the running apps.



Ask AI

- **Static web files:** refreshes the browser.
- **Dependencies:** loads them in place. The server restarts only for native assets. Flutter apps hot restart for pure Dart packages and relaunch for native ones.
- **Flutter assets and fonts:** relaunches the app.
- **Configuration:** not watched, so press **R** after editing `config/`.

The **R** key adapts to the situation. In watch mode, where saves already hot reload, **R** performs a hot restart: use it when a change cannot be hot reloaded, such as code that only runs at startup. If the project has errors instead, the session stays open and tells you what failed; the server boots automatically once a save fixes the errors, or press **R** to retry the build and start.

To start without watching, pass `--no-watch`. Nothing reloads on save; there, **R** hot reloads on demand, and **Shift+R** restarts the server.

Manage migrations from the terminal

Every time `serverpod start` boots the server, including restarts, it applies any pending migrations. While it runs, you handle further schema changes from the terminal without leaving the session:

- **M** creates a migration from your current model changes and applies it.
- **P** creates a repair migration to reconcile a database that has drifted from your migrations, and applies it. This shortcut is not shown in the bottom bar; press **H** to see it listed.
- **A** applies pending migrations. Use it to retry after a failed apply.

Hold **Shift** with **M** or **P** to force the migration past Serverpod's warning that information may be destroyed, so force deliberately. **Shift+M** still creates nothing when your models have no changes. For an empty migration, run `serverpod create-migration --empty`. **Shift+P** creates a repair migration even when the database has not drifted. For how migrations work, see [Migrations](#).

Choose a run mode

By default, `serverpod start` runs in the `development` run mode. To start in another mode, forward a `--mode` argument to the server after `--`:

```
serverpod start -- --mode staging
```

The run mode selects which configuration and passwords the server loads. See [Run modes](#) for what each mode reads. Flutter apps only launch in the `development` run mode.

Choose which Flutter apps start

The `serverpod: flutter_apps:` block in the server's `pubspec.yaml` lists the Flutter apps that `serverpod start` can run. Each one has its own entry, keyed by an id you choose. New projects with a Flutter app include this block:

```
serverpod:
  flutter_apps:
    my_project:
      path: ../my_project_flutter
      displayName: "My project app"
      auto_launch: true
      target: lib/driver.dart
```

Each entry accepts these keys:

Key	Description
<code>path</code>	Required. The Flutter package folder, relative to the server package.
<code>displayName</code>	The name on the app's tab. Defaults to the app id.
<code>auto_launch</code>	Set to <code>true</code> to launch the app when the session starts. Defaults to <code>false</code> .
<code>device</code>	The <code>flutter run -d</code> device, for example <code>chrome</code> or <code>macos</code> . Without it, the app runs on Flutter's web server and opens in your browser.
Any other key	Passed to <code>flutter run</code> as a flag. For example, <code>target: lib/driver.dart</code> becomes <code>--target=lib/driver.dart</code> , and <code>release: true</code> becomes <code>--release</code> .

Apps without `auto_launch: true` stay stopped when the session begins. To run one, press **Ctrl+R** in the `serverpod start` terminal and pick it from the app launch panel.

Without a `flutter_apps` block, `serverpod start` launches the `<project>_flutter` package beside your server, when that package exists.

In watch mode, adding or removing an app takes effect when you save. A running app keeps its old settings until you restart it from the same panel.

Run the server on its own

By default, `serverpod start` also runs the Flutter apps [set to launch automatically](#). To start only the server, disable that:

```
serverpod start --no-flutter
```

You can still launch an app on demand: press **Ctrl+R** in the terminal to open the app launch panel.

If your project has a Docker Compose file (`docker-compose.yaml`, `compose.yaml`, or another standard name), `serverpod start` brings it up automatically when your database is a Postgres on `localhost` with no `dataPath`, and stops the services it started when the session ends. When that does not apply, for example an embedded or remote database with [Redis](#) in Compose, pass `--docker` to start the stack anyway, or `--no-docker` to keep it off.

To run without the interactive terminal, pass `--no-tui`. When the output is not a terminal, for example in CI, `serverpod start` falls back to plain output on its own.

Run the server directly

Outside `serverpod start`, the server is a plain Dart program you can run yourself from the server package directory:

```
dart run bin/main.dart --apply-migrations
```

The server accepts arguments that control how it starts: `--mode` selects the run mode, `--role` selects the [server role](#), and `--apply-migrations` applies pending migrations during startup. Without `--apply-migrations`, the server still checks the database schema at startup. On a mismatch, it exits with code 1 in the `development` run mode and logs a warning in other run modes. See the [run options reference](#) for the full list, and the [Deploy](#) section for running in production.

Choose a server role

In production, the `--role` argument controls which parts of the server run:

- `monolith` (default) runs everything: the API, Insights, and web servers, plus [future calls](#) and [health checks](#).
- `serverless` serves requests only. Future calls and [health metric collection](#) are disabled, which fits platforms that start and stop instances on demand. The health probes still answer.
- `maintenance` starts no servers. It performs one-shot work and exits, which fits CI jobs and scheduled maintenance tasks. With `--apply-migrations` or `--apply-repair-migration`, it applies the migrations and does nothing else. Without those flags, it runs any due future calls and one round of

health checks. Outside `development`, a failed migration is only logged, so the process can still exit with code 0.

Run code on shutdown

Register a shutdown task to do cleanup work when the server stops, such as flushing state or releasing an external resource. Tasks run after the server stops accepting requests but before the database and Redis connections close, so they can still use them.

WARNING

Shutdown tasks are an experimental API and can change in a breaking way in any minor release.

Register them on the `Serverpod` instance in `lib/server.dart`, where you create it:

```
pod.experimental.shutdownTasks.addTask(#taskIdentifier, () async {  
  // Your shutdown logic here.  
});
```

Each task is registered under an identifier you choose, used in log messages and to remove the task again. Any object works; `#taskIdentifier` above is a Dart symbol, which is a convenient way to write a constant name.

```
pod.experimental.shutdownTasks.removeTask(#taskIdentifier);
```

Registering two tasks under the same identifier throws a `StateError`. All tasks run concurrently, and the server waits for them all before shutting down. A task that throws does not stop the shutdown, but the error is logged and the process exits with a non-zero code, which a host reading exit status will treat as a failed shutdown.

Related

- [serverpod start](#) reference: every command-line option.
- [Configuration](#): the run modes and files the server loads on start.
- [Migrations](#): how schema changes reach the database.
- [Deploy to Serverpod Cloud](#): run your server in production instead of locally.